

Integración Continua (CI) de un Microservicio Flask con AWS CodeBuild

Taller 2 - Actividades DevOps

Autor

Breitner Enrique Gonzalez Angarita

Prof.

Mario José Villamizar Cano

Universidad de los Andes

26 de abril de 2026

Introducción

En este taller implementé un pipeline de Integración Continua (CI) desde cero usando AWS CodeBuild para manejar la construcción, pruebas y despliegue automático de un microservicio Flask. La idea principal de este ejercicio es simple: eliminar tareas repetitivas del desarrollo y dejar que las máquinas hagan lo que hacen mejor: ejecutar procesos de forma consistente y confiable. Al final del taller, tengo un sistema donde cada cambio de código es validado automáticamente, testeado, empaquetado y listo para desplegar.

Hoy en día, es difícil imaginar un desarrollador serio que no use CI/CD. Los beneficios son claros: detectar problemas antes de que lleguen a producción, garantizar que cada despliegue ha pasado todas las validaciones, y poder lanzar cambios rápidamente sin miedo a romper algo. Este taller me muestra exactamente cómo se hace en la práctica.

Objetivos

- Comprender los conceptos fundamentales de Integración Continua y Despliegue Continuo
- Configurar un proyecto en AWS CodeBuild para compilación automatizada
- Implementar archivos `buildspec.yml` para definir fases de construcción
- Automatizar pruebas unitarias con `pytest` y análisis de cobertura
- Desplegar aplicaciones automáticamente en Elastic Beanstalk tras validación

Conceptos de CI/CD

CI/CD cambió fundamentalmente la forma en que trabajamos. Antes, compilar código, ejecutar pruebas y desplegar a producción era un proceso lento y propenso a errores que podía llevar días. Ahora, todo sucede automáticamente en minutos. Menos errores humanos, más velocidad, mejor calidad de código. Es un cambio de mentalidad que valió la pena entender.

Integración Continua (CI)

En Integración Continua, cada desarrollador sube su código al repositorio central varias veces al día. En lugar de esperar semanas para fusionar cambios (cuando inevitablemente hay conflictos), los cambios se integran constantemente. Esto significa que los problemas se detectan rápidamente, antes de que se conviertan en un dolor de cabeza mayor.

- Los desarrolladores realizan commits pequeños y frecuentes (varias veces al día)
- Cada commit dispara una construcción automatizada en el servidor CI
- Las pruebas automatizadas se ejecutan inmediatamente para validar el cambio
- Los desarrolladores son notificados instantáneamente de fallos en la compilación

Despliegue Continuo (CD)

El Despliegue Continuo es el siguiente paso: si el código pasa todas las pruebas, se despliega automáticamente a producción. Suena arriesgado al principio, pero con las pruebas correctas en lugar, es más seguro que el despliegue manual. Los usuarios ven nuevas funcionalidades y correcciones de bugs mucho más rápido.

- El código se despliega automáticamente a producción después de pasar todas las pruebas
- Reduce significativamente el tiempo entre desarrollo y usuarios finales
- Minimiza riesgos mediante validación continua y feedback rápido

Configuración de AWS CodeBuild

AWS CodeBuild es donde toda la magia sucede. Es el motor que ejecuta tu `buildspec.yml`, compilando el código, corriendo las pruebas y generando artefactos. Lo mejor es que no tienes que preocuparte por mantener servidores de construcción; AWS se encarga de eso por ti. Solo defines qué quieres que haga y CodeBuild lo hace.

Crear Proyecto en CodeBuild

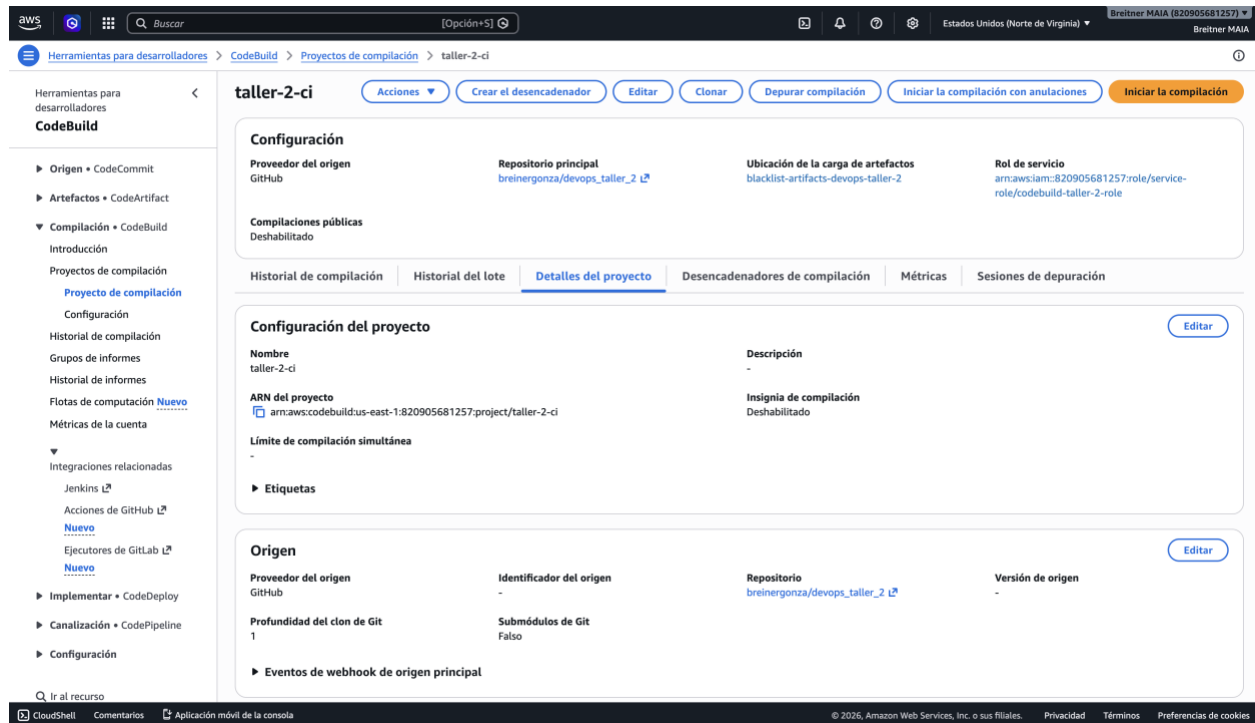
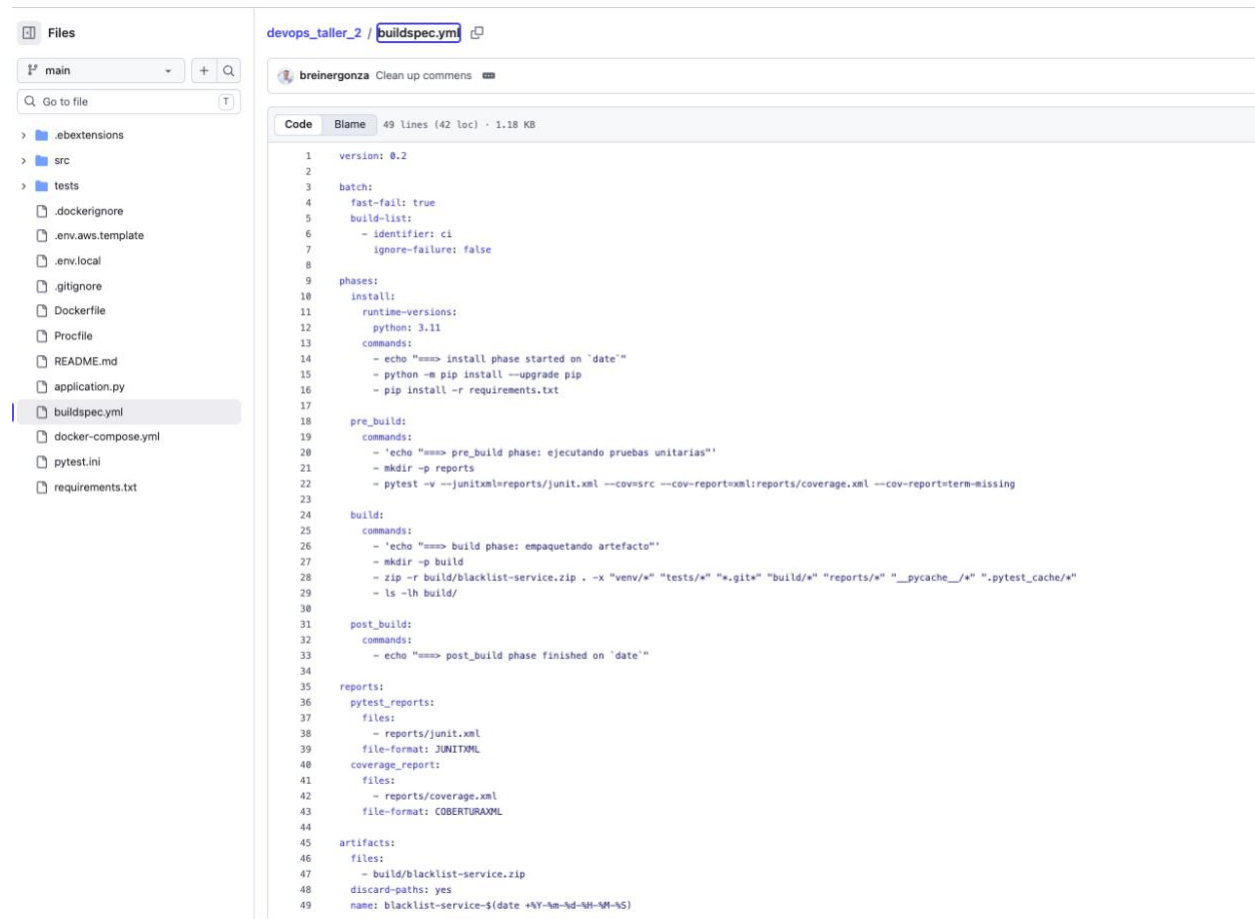


Ilustración 1. AWS CodeBuild Console - taller-2-cli

- **Nombre:** taller-2-ci
- **Fuente:** Repositorio GitHub (https://github.com/breinergonza/devops_taller_2)
- **Entorno:** Imagen administrada con Python 3.9 y Amazon Linux 2
- **Rol IAM:** Permisos para EC2, RDS, S3 y CloudWatch

Archivo buildspec.yml

El archivo **buildspec.yml** es el blueprint del pipeline. Es donde le decimos a CodeBuild exactamente qué hacer en cada paso. Cada fase (install, pre_build, build, post_build) tiene un propósito: una es para instalar dependencias, otra para correr tests, otra para compilar, y la última para desplegar. Get it right en este archivo y todo funciona; get it wrong y todo falla.



```
1 version: 0.2
2
3 batch:
4   fast-fail: true
5   build-list:
6     - identifier: ci
7     ignore-failure: false
8
9 phases:
10  install:
11    runtime-versions:
12      python: 3.11
13    commands:
14      - echo "=== install phase started on 'date'"
15      - python -m pip install --upgrade pip
16      - pip install -r requirements.txt
17
18  pre_build:
19    commands:
20      - 'echo "=== pre_build phase: ejecutando pruebas unitarias"'
21      - mkdir -p reports
22      - pytest -v --junitxml=reports/junit.xml --cov=src --cov-report=xml:reports/coverage.xml --cov-report=term-missing
23
24  build:
25    commands:
26      - 'echo "=== build phase: empaquetando artefacto"'
27      - mkdir -p build
28      - zip -r build/blacklist-service.zip . -x "venv/*" "tests/*" "*.git*" "build/*" "reports/*" "__pycache__/*" ".pytest_cache/*"
29      - ls -lh build/
30
31  post_build:
32    commands:
33      - echo "=== post_build phase finished on 'date'"
34
35  reports:
36    pytest_reports:
37      files:
38        - reports/junit.xml
39      file-format: JUNITXML
40    coverage_report:
41      files:
42        - reports/coverage.xml
43      file-format: COBERTURAXML
44
45  artifacts:
46    files:
47      - build/blacklist-service.zip
48    discard-paths: yes
49    name: blacklist-service-${date +%Y-%m-%d-%H-%M-%S}
```

Ilustración 2. Archivo buildspec.yml y sus fases

- Fase install: Instala dependencias del sistema y Python (pip install -r requirements.txt)
- Fase pre_build: Configura variables de entorno, conecta a base de datos, valida credenciales
- Fase build: Ejecuta pruebas unitarias (pytest) y genera reportes de cobertura
- Fase post_build: Compila artefactos en ZIP, sube a S3, notifica Elastic Beanstalk

Pruebas Automatizadas

Las pruebas automatizadas son el corazón de un pipeline CI/CD confiable. Validan que el código funciona como se espera antes de que llegue a producción, capturando regresiones tempranamente.

Suite de Pruebas con pytest

```
130 pre_build phase: ejecutando pruebas unitarias
131
132 [Container] 2026/04/26 03:26:23.823009 Running command mkdir -p reports
133
134 [Container] 2026/04/26 03:26:23.832014 Running command pytest -v --junitxml=reports/junit.xml --cov=src --cov-report=xml:reports/coverage.xml --cov-report=term-missing
135
136 ===== test session starts =====
137 platform linux -- Python 3.11.15, pytest-7.4.4, pluggy-1.6.0 -- /root/.pyenv/versions/3.11.15/bin/python3.11
138 cachedir: .pytest_cache
139 rootdir: /codebuild/output/src3331234537/src/github.com/breinergonza/devops_taller_2
140 configfile: pytest.ini
141 testpaths: tests
142 plugins: cov-4.1.0
143 collecting ... collected 12 items
144
145 tests/test_blacklist_get.py::test_get_blacklist_found PASSED [ 8%]
146 tests/test_blacklist_get.py::test_get_blacklist_not_found PASSED [ 16%]
147 tests/test_blacklist_get.py::test_get_blacklist_unauthorized PASSED [ 25%]
148 tests/test_blacklist_get.py::test_get_blacklist_returns_latest_reason PASSED [ 33%]
149 tests/test_blacklist_post.py::test_post_blacklist_ok PASSED [ 41%]
150 tests/test_blacklist_post.py::test_post_blacklist_without_blocked_reason PASSED [ 50%]
151 tests/test_blacklist_post.py::test_post_blacklist_unauthorized_without_token PASSED [ 58%]
152 tests/test_blacklist_post.py::test_post_blacklist_unauthorized_with_bad_token PASSED [ 66%]
153 tests/test_blacklist_post.py::test_post_blacklist_missing_required_field PASSED [ 75%]
154 tests/test_blacklist_post.py::test_post_blacklist_blocked_reason_too_long PASSED [ 83%]
155 tests/test_health.py::test_index_returns_metadata PASSED [100%]
156
157 - generated xml file: /codebuild/output/src3331234537/src/github.com/breinergonza/devops_taller_2/reports/junit.xml -
```

Ilustración 3. Salida de pytest con pruebas exitosas.

- Framework: pytest con fixtures para cliente Flask test
- Pruebas unitarias de endpoints REST GET y POST
- Validación de autenticación con tokens Bearer
- Pruebas de integración con base de datos PostgreSQL

Reporte de Cobertura

```
159 ----- coverage: platform linux, python 3.11.15-final-0 -----
160 Name                               Stmts  Miss  Cover   Missing
161 -----
162 src/__init__.py                      0      0  100%
163 src/main.py                          29      0  100%
164 src/models/__init__.py               0      0  100%
165 src/models/blacklist.py              12      1   92%   22
166 src/schemas/__init__.py             0      0  100%
167 src/schemas/blacklist_schema.py     10     10   0%    2-17
168 src/views/__init__.py                0      0  100%
169 src/views/blacklist_view.py          39      1   97%   65
170 -----
171 TOTAL                                90     12   87%
172 Coverage XML written to file reports/coverage.xml
173
174 ===== 12 passed in 0.34s =====
```

Ilustración 4. Reporte de cobertura con el porcentaje de líneas cubiertas.

- Herramienta: pytest-cov para análisis de cobertura
- Objetivo: Mantener cobertura mínima del 80% de líneas ejecutadas
- Reporte generado en formato HTML para análisis detallado

Ejecución del Pipeline

Construcción Exitosa

The screenshot shows the AWS CodeBuild console for a project named 'taller-2-ci'. The left sidebar contains navigation links for 'Herramientas para desarrolladores', 'CodeBuild', and various project actions. The main panel displays the 'Configuración' (Configuration) tab, showing the provider (GitHub), repository (breitnergonza/devops_taller_2), and build location (blacklist-artifacts-devops-taller-2). Below this, the 'Historial de compilación' (Build History) tab is active, showing a table of build attempts. A red box highlights the build attempt with ID 'taller-2-ci:3ac933e6-1aff-4cca-9fcd-9d6f45a80e44', which has a status of 'Error'. A red arrow points from the text 'Error en escenario controlado' to this row. The table columns are: Ejecución del lote, Estado, Número de lote, Versión de origen, Remitente, Duración, and Completado.

Ejecución del lote	Estado	Número de lote	Versión de origen	Remitente	Duración	Completado
taller-2-ci:ad8cb0e0-bbf2-4cb4-bb5e-7eb0a79b7669	Realizado correctamente	11	fd899cf32943cd2b5a56865cd336f79d5060e25	GitHub-Hookshot/03a48eb	2 minutos 5 segundos	hace 8 minutos
taller-2-ci:07b28311-a626-46ab-bd9e-073d520538a2	Realizado correctamente	10	686a9a1ec967023cee311122e5d15fe752949b51	GitHub-Hookshot/03a48eb	2 minutos 5 segundos	hace 4 horas
taller-2-ci:9944676f-0075-43ac-b559-ef9b6d115672	Realizado correctamente	9	27b73c3fe234857a2b7fb9ad9c74a2e482ab18c2	GitHub-Hookshot/03a48eb	2 minutos 5 segundos	hace 18 horas
taller-2-ci:3ac933e6-1aff-4cca-9fcd-9d6f45a80e44	Error	8	0c59e841f0a62c943b9065a54df1774673709c1	GitHub-Hookshot/03a48eb	2 minutos 5 segundos	hace 18 horas
taller-2-ci:ccb923cb-0295-4e2d-93f8-367f072c1a1d	Realizado correctamente	7	6978ea709172d7c06d1d2fa64b91046cd3bca130	GitHub-Hookshot/03a48eb	2 minutos 5 segundos	hace 19 horas

Ilustración 5. Codebuild con pipeline de pruebas exitosas

Una construcción exitosa indica que todo el código ha sido validado y es seguro para despliegue.

- Duración típica: 5-8 minutos para compilación y pruebas
- Todas las fases completadas: install → pre_build → build → post_build
- Artefactos generados: ZIP con código compilado listo para despliegue

Manejo de Fallos

taller-2-ci:3ac933e6-1aff-4cca-9fcd-9d6f45a80e44

Detener lote Eliminar lote Volver a intentar el lote

Estado del lote

Estado Error

Remitente GitHub-Hookshot/03a48eb

ARN del lote arn:aws:codebuild:us-east-1:820905681257:build-batch/taller-2-ci:3ac933e6-1aff-4cca-9fcd-9d6f45a80e44

Versión de origen 0c59e841f0a62c943b9065a54df1f774673709c1

Hora de inicio hace 19 horas

Hora de finalización hace 19 horas

Número de compilación 8

Compilaciones en el lote Detalles de la fase Informes Variables del entorno Detalles del lote

Compilaciones en el lote

Ver los artefactos

Identificador de grupo	Ejecución de la compilación	Estado	Depende de	Solicitado el
DOWNLOAD_SOURCE	taller-2-ci:b22e279a-d726-4c31-b9b5-9935c9f1720b	Realizado correctamente	-	hace 19 horas
ci	taller-2-ci:f3f74ad8-5bc4-49cc-a05d-b035ace0a4a	Error	-	hace 19 horas

Ilustración 6. Paso en el que falló la integración continua con CodeBuild.

aws

Buscar [Opción=5]

Estados Unidos (Norte de Virginia) Bretnner MAIA (B20905681257) Bretnner MAIA

Herramientas para desarrolladores > CodeBuild > Proyectos de compilación > taller-2-ci > taller-2-ci:f3f74ad8-5bc4-49cc-a05d-b035ace0a4a

Herramientas para desarrolladores

CodeBuild

Origen • CodeCommit

Artefactos • CodeArtifact

Compilación • CodeBuild

Introducción

Proyectos de compilación

Proyecto de compilación

Configuración

Historial de compilación

Grupos de informes

Historial de informes

Flotas de computación Nuevo

Métricas de la cuenta

Integraciones relacionadas

Jenkins

Acciones de GitHub

Nuevo

Ejecutores de GitLab

Nuevo

Implementar • CodeDeploy

Canalización • CodePipeline

```
147 tests/test_blacklist_get.py::test_get_blacklist_returns_latest_reason PASSED [ 33%]
148 tests/test_blacklist_post.py::test_post_blacklist_ok FAILED [ 43%]
149 tests/test_blacklist_post.py::test_post_blacklist_without_blocked_reason PASSED [ 50%]
150 tests/test_blacklist_post.py::test_post_blacklist_unauthorized_without_token PASSED [ 58%]
151 tests/test_blacklist_post.py::test_post_blacklist_unauthorized_with_bad_token PASSED [ 66%]
152 tests/test_blacklist_post.py::test_post_blacklist_missing_required_field PASSED [ 73%]
153 tests/test_blacklist_post.py::test_post_blacklist_blocked_reason_too_long PASSED [ 83%]
154 tests/test_health.py::test_health_check_returns_200 PASSED [ 91%]
155 tests/test_health.py::test_index_returns_metadata PASSED [100%]

156 ----- FAILURES -----
157 test_post_blacklist_ok
158 tests/test_blacklist_post.py:16: in test_post_blacklist_ok
159     assert body["email"] == "test@uniandes.edu.co"
160 E   AssertionError: assert 't@leuniandes.edu.co' == 'test@uniandes.edu.co'
161 E   - test@uniandes.edu.co
162 E   ? AAAA
163 E   + t@leuniandes.edu.co
164 E   ? AAA
165 E   ? AAA
166 generated xml file: /codebuild/output/src626866186/src/github.com/breinergonza/devops.taller_2/reports/junit.xml
167
168 ----- coverage: platform linux, python 3.11.15-Final-0 -----
169
170 Name                               Stmts   Miss  Cover   Missing
171 -----
172 src/__init__.py                     0      0  100%
173 src/main.py                         29      0  100%
174 src/models/_init_.py                0      0  100%
175 src/models/blacklist.py             12      1    92%   22
176 src/schemas/_init_.py              0      0  100%
177 src/schemas/blacklist_schema.py    10     10    0%   2-17
178 src/views/_init_.py                 0      0  100%
179 src/views/blacklist_view.py         39      1    97%   65
180 -----
181 TOTAL                               90     12    87%
182 Coverage XML written to file reports/coverage.xml
183
184 short test summary info
185 FAILED tests/test_blacklist_post.py::test_post_blacklist_ok - AssertionError:...
186 1 failed, 11 passed in 0.36s
187
188 [Container] 2026/04/26 03:22:38.086990 Command did not exit successfully pytest -v --junitxml=reports/junit.xml --cov=src --cov-report=xml:reports/coverage.xml --cov-report-term=missing exit status 1
189 [Container] 2026/04/26 03:22:38.091999 Phase complete: PRE_BUILD State: FAILED
190 [Container] 2026/04/26 03:22:38.092015 Phase context status code: COMMAND_EXECUTION_ERROR Message: Error while executing command: pytest -v --junitxml=reports/junit.xml --cov=src --cov-report=xml:reports/coverage.xml --cov-report-term=missing. Reason: exit status 1
```

Ilustración 7. Paso en el que fallaron las pruebas automáticas.

Cuando una construcción falla, el pipeline se detiene inmediatamente previniendo despliegues de código defectuoso.

- Notificaciones automáticas enviadas al equipo de desarrollo
- Logs detallados disponibles en CloudWatch para investigación
- El código NO se despliega automáticamente si las pruebas fallan

Gestión de Variables de Entorno

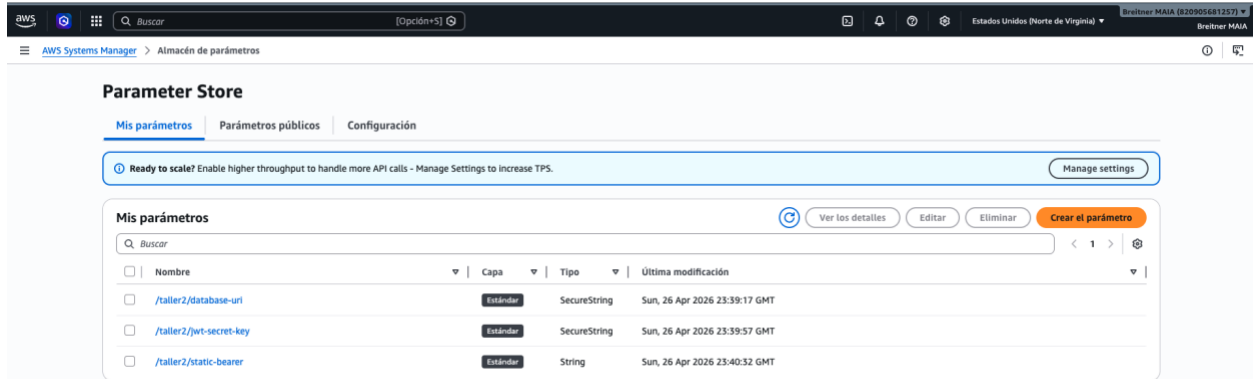


Ilustración 8. Variables de sesión en AWS System Manager.

AWS Systems Manager proporciona una ubicación centralizada y segura para almacenar variables sensibles como credenciales de base de datos.

- DATABASE_URI: Cadena de conexión a RDS PostgreSQL
- JWT_SECRET_KEY: Clave secreta para generación de tokens
- STATIC_BEARER: Token de acceso para autenticación estática

Mejores Prácticas

- Lo ideal es realizar commits pequeños y frecuentes con mensajes descriptivos
- Mantener cobertura de pruebas mínima del 80% en la pipeline
- Incluir validaciones de seguridad en la pipeline (SAST, dependency scanning)
- Documentar y practicar procedimientos de rollback ante despliegues problemáticos

Conclusiones

A través de este taller pude explorar cómo implementar un pipeline de integración continua robusto usando AWS CodeBuild. EL objetivo fue pasar de un concepto teórico sobre CI a materializar la teoría con la práctica mediante la configuración de un proyecto real que automatiza las tareas de compilación, testing y despliegue de mi microservicio Flask. Esta automatización no es un lujo, sino una necesidad en los equipos de desarrollo actuales.

Un aspecto clave que aprendí es que la configuración del `buildspec.yml` es el corazón del pipeline. Las pequeñas fallas sintácticas o de lógica en este archivo pueden hacer que todo el proceso fracase, como evidencié durante la depuración de errores YAML. Esto subraya la importancia de validar y testear la configuración antes de ponerla en producción.

Las pruebas automatizadas con `pytest` demostraron ser cruciales para mantener la calidad del código. Un porcentaje de cobertura de código alto no garantiza la ausencia de bugs, pero reduce significativamente el riesgo. La capacidad de ejecutar estas pruebas automáticamente en cada cambio me permite detectar regresiones casi instantáneamente.

Finalmente, la integración con servicios de AWS como S3 y Elastic Beanstalk demuestra cómo un pipeline de CI no existe de manera aislada y debe conectarse con el ecosistema de despliegue. La capacidad de generar artefactos, almacenarlos en S3 y desplegarlos automáticamente a Elastic Beanstalk cierra el ciclo completo del DevOps. Para mi la curva de aprendizaje fue un poquito alta ya que solo tenía conocimientos en infraestructura en AWS sin embargo con cada semana es posible profundizar en el ecosistema de desarrollo de esta nube.

Referencias

- AWS. (2026). Documentación de AWS CodeBuild. Recuperado de <https://docs.aws.amazon.com/codebuild/>
- AWS. (2026). Documentación de AWS CodePipeline. Recuperado de <https://docs.aws.amazon.com/codepipeline/>
- AWS. (2026). Documentación de Elastic Beanstalk. Recuperado de <https://docs.aws.amazon.com/elasticbeanstalk/>